VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU | CEX Listing | Marketing

MADE IN CANADA

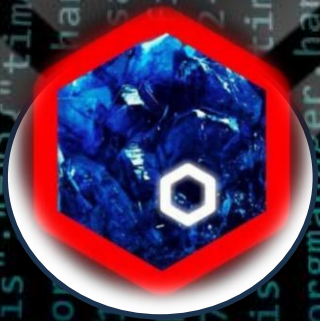
CRYSTAL STONES

AUDIT

SECURITY ASSESSMENT

28th August 2026

For



Making Blockchain, Defi And Web3 A Safer Place.

Smart Check

SLITHER

TRAIL OF BITS

MythX

@VB_Audit @Vitalblock @VB-Audit www.vitalblock.org

Table Of Content

Chapter 1 Introduction

Chapter 2 Findings

INTRODUCTION

Auditing Firm	VITAL BLOCK SECURITY
Client Firm	CRYSTAL STONES
Methodology	Automated Analysis, Manual Code Review
Language	Solidity
Contract Address	0xe252FCb1Aa2E0876E9B5f3eD1e15B9b4d11A0b00
Source Code Light	Public Source
Centralization	Active ownership
Blockchain	BINANCE CHAIN
Dependencies	OpenZeppelin Contracts (v5+ compatible)
Compiler	Solidity v0.8.4+commit.c7e474f2
Website	www.crystalstones.org/
Telegram	@CrystalStones
Twitter / X	@crystalstones01
Prelim Report Date	August 26TH 2026
Final Report Date	August 28TH 2026

Verify the authenticity of this report on our GitHub Repo: https://www.github.com/vital-block

Document Properties

Client	CRYSTAL STONES
Title	Smart Contract Audit Report
Target	CRYSTAL STONES
Audit Version	1.0
Author	Akhmetshin Marat
Auditors	Akhmetshin Marat, James BK, Benny Matin
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	AUGUST 27th, 2026	James BK	Final Released
1.0-AP	AUGUST 28th , 2026	Jimmy Cole	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone	+44 7944 248057
Email	info@vitalblock.org

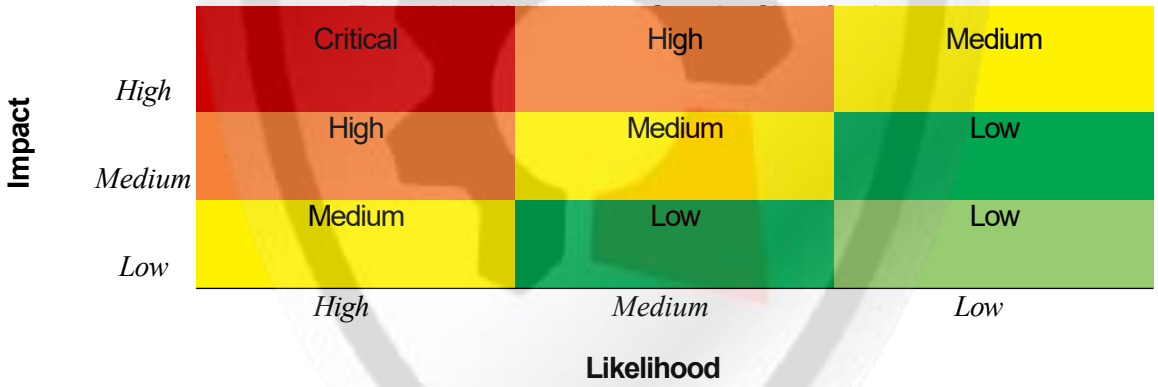
In the following, we show the specific pull request and the commit hash value used in this audit.

- CRYSTAL STONES (KP59750)
- CRYSTAL STONES (CRYSTAL STONES) | BEP-20 | Address: 0xe252FCb1...4d11A0b00 | BscScan (HOPET50)

About Vital Block Security

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We can audit custom smart contracts, DApps, Rust, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/vitalblock),

Twitter / X: (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).



Methodology (1)

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology [4]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

SCOPE OF WORK

Vital Block Security will conduct the smart contract audit of its Sol source code. The audit scope of work is strictly limited to mentioned .SOL file only.

O.CRYSTALSTONE.sol

External contracts and/or interfaces dependencies are not checked due to being out of scope.

Verify audited contract code Repo.

PUBLIC CONTRACTt ADDRESS:
0xe252FCb1Aa2E0876E9B5f3eD1e15B9b4d11A0b00

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block Security auditing process and methodology:

CONNECT

- **The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.**

AUDIT

- **Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:**
  - **Remix IDE Developer Tool**
  - **Open Zeppelin Code Analyzer**
  - **SWC Vulnerabilities Registry**
  - **DEX Dependencies, e.g., Pancakeswap, Uniswap**
- **Simulations are performed to identify centralized exploits causing contract and/or trade locks.**
- **A manual line-by-line analysis is performed to identify contract issues and centralized privileges. We may inspect below mentioned common contract vulnerabilities, and centralized exploits:**

Centralized Exploits	**Token Supply Manipulation** **Access Control and Authorization** **Assets Manipulation** **Ownership Control** **Liquidity Access** **Stop and Pause Trading** **Ownable Library Verification**

Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.

Table 1.0 The Full Audit Checklist

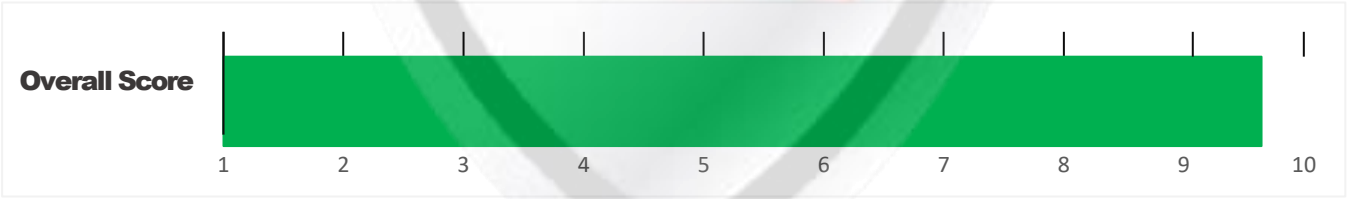
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the automated and manual analysis of the CRYSTAL STONE Sol code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical !	Major "	Medium #	Minor $	Unknown %
Open	0	0	0	0	2
Acknowledged	1	0	3	0	1
Resolved	1	0	1	0	0
Noteworthy OnlyOwner Privileges	Set Taxes and Ratios, Airdrop, Set Protection Settings, Set Reward Properties, Set Reflector Settings, Set Swap Settings, Set Pair and Router				

CRYSTAL STONE Smart contract has achieved the following score: 95.0 %



Please note that smart contracts deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- **Privileged roles can be granted the power to** pause() **the contract in case of an external attack.**
- **Privileged roles can use functions like,** include(), **and** exclude() **to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.**

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- **The client can lower centralization-related risks by implementing below mentioned practices:**
- **Privileged role's private key must be carefully secured to avoid any potential hack.**
- **Privileged role should be shared by multi-signature (multi-sig) wallets.**
- **Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.**
- **Renouncing the contract ownership, and privileged roles.**
- **Remove functions with elevated centralization risk.**

Understand the project's initial asset distribution. Assets in the liquidity pair should be locked. Assets outside the liquidity pair should be locked with a release schedule.

RISK CATEGORIES

Smart contracts are generally designed to hold, approve, and transfer tokens. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical !	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major "	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium #	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor $	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown %	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.

OPTIMIZATIONS | CRYSTAL STONES

ID	Title	Category	Status
CTV	**Logarithm Refinement Optimization**	Gas Optimization	Acknowledged
COP	**Checks Can Be Performed Earlier**	Gas Optimization	Acknowledged
CDP	**Unnecessary Use Of SafeMath**	Gas Optimization	Acknowledged
CWY	**Struct Optimization**	Gas Optimization	Acknowledged
CGT	**Unused State Variable**	Gas Optimization	Acknowledged

Executive Summary

An audit was conducted on the compiled source code of the **CRYSTAL STONES** token contract deployed on the BNB Smart Chain. The contract utilizes the FatToken standard pattern Overall risk score is classified as **MEDIUM-HIGH** due to administrative privilege concentration and typical automated DEX router dependency risks.

Key Findings

Overall, these contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), 1 High-severith, 2 medium-severity vulnerabilities, 1 low-severity vulnerabilities, and 1 informational recommen- dations.

Table 2.1: Key **CRYSTAL STONE** **Audit Findings**

File / Modules Analyzed	Function / Purpose	Risk Classification
Context.sol	Context execution utilities (_msgSender(), _msgData())	PASS (Zero Risk)
TokenDistributor.sol	Helper contract for token processing & free conversions	LOW (Operational)
Ownable.sol	Standard access control & ownership modifier logic	MEDIUM (Centralization)
Crystal Stone.sol	BEP-20 token core logic, fee mechanisms, and swap router calls	MEDIUM HIGH (Privilege/Slippage)

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to page 10 for details.

AUTOMATED ANALYSIS

Symbol	Definition
	Function modifies state
	Function is payable
	Function is internal
	Function is private
	Function is important

** **CRYSTALSTONE** **	Interface			
	└	totalSupply	External !		!		NO!	
	└	decimals	External !		!		NO!	
	└	symbol	External !		!		NO!	
	└	name	External !		!		NO!	
	└	getOwner	External !				NO!	
	└	balanceOf	External !		!		NO!	
	└	transfer	External !	"	!		|NO!	
	└	allowance	External !		!		NO!	
	└	approve	External !	"	!		|NO!	
	└	transferFrom	External !	"			NO!	
	IFactoryV2	Interface					
	└	getPair	External !				NO!	
	└	createPair	External !	"			NO!	
	IV2Pair	Interface					
	└	factory	External !				NO!	
	└	getReserves	External !				NO!	
	└	sync	External !	"			NO!	

IRouter01	Interface			
└	factory	External ❗	|NO❗
└	WBNB	External ❗	|NO❗
└	addLiquidityWBNB	External ❗	# |NO❗
└	addLiquidity	External ❗	" |NO❗
└	swapExacWBNBForTokens	External ❗	# |NO❗
└	getAmountsOut	External ❗	|NO❗
└	getAmountsIn	External ❗	|NO❗
IRouter02	Interface	IRouter01		
└	swapExactTokensForWBNBSupportingFeeOnTransferTokens	External ❗	" |NO❗
└	swapExactWBNBForTokensSupportingFeeOnTransferTokens	External ❗	# |NO❗
└	swapExactTokensForTokensSupportingFeeOnTransferTokens	External ❗	" ! 🛑 |NO❗
└	swapExactTokensForTokens	External ❗	" |NO❗
Protections	Interface			
└	checkUser	External ❗	" ! 🛑 |NO❗
└	setLaunch	External ❗	" ! 🛑 |NO❗
└	setLpPair	External ❗	" ! 🛑 |NO❗
└	CRYSTALSTONE	External ❗	" |NO❗
└	removeSniper	External ❗	" |NO❗
Cashier	Interface			
└	setRewardsProperties	External ❗	" |NO❗
└	tally	External ❗	" |NO❗
└	load	External ❗	# |NO❗
└	cashout	External ❗	" |NO❗
└	giveMeWelfarePlease	External ❗	" |NO❗
└	getTotalDistributed	External ❗	|NO❗
└	getUserInfo	External ❗	|NO❗
└	getUserRealizedRewards	External ❗	|NO❗

| └ | getPendingRewards | External ❗ | | |NO❗ |

| └ | initialize | External ❗ | " | |NO❗ |

| └ | getCurrentReward | External ❗ | | |NO❗ |

||||||

| **WBNB** | Implementation | SafeMath |||

| └ | <Constructor> | Public ❗ | | # |NO❗ |

| └ | transferOwner | External ❗ | " | | onlyOwner |

| └ | renounceOwnership | External ❗ | " | | NO❗|

| └ | setOperator | Public ❗ | " | |NO❗ |

| └ | renounceOriginalDeployer | External ❗ | " | |NO❗ |

| └ | <Receive Ether> | External ❗ | | # |NO❗ |

| └ | totalSupply | External ❗ | | |NO❗ |

| └ | decimals | External ❗ | | |NO❗ |

| └ | symbol | External ❗ | | |NO❗ |

| └ | name | External ❗ | | |NO❗ |

| └ | getOwner | External ❗ | | ! |NO❗ |

| └ | balanceOf | Public ❗ | | ! |NO❗ |

| └ | allowance | External ❗ | | ! |NO❗ |

| └ | approve | External ❗ | " ! 🛑 |NO❗ |

| └ | _approve | Internal $ | " | | |

| └ | approveContractContingency | Public ❗ | " ! 🛑 | onlyOwner |

| └ | transfer | External ❗ | " | |NO❗ |

| └ | transferFrom | External ❗ | " | |NO❗ |

| └ | setNewRouter | External ❗ | " | | onlyOwner |

| └ | setLpPair | External ❗ | " | | onlyOwner |

| └ | setInitializers | External ❗ | " | | onlyOwner |

| └ | isExcludedFromFees | External ❗ | | |NO❗ |

| └ | isExcludedFromDividends | External ❗ | | |NO❗ |

| └ | isExcludedFromProtection | External ❗ | | |NO❗ |

| └ | setDividendExcluded | Public ❗ | " | | onlyOwner |

| └ | setExcludedFromFees | Public ❗ | " | | onlyOwner |

CN-01 Key Findings

Category	Severity	Target	Status
Business Logic	HIGH	Owner Privileges	Acknowledged

In Centralization Risks & Single Point of Failure (Owner Privileges)

Vulnerability Analysis:

The contract inherits from Ownable. The onlyOwner modifier protects administrative mechanisms including setting fee percentages (tax configuration), modifying DEX router pairs, setting max transaction amounts/wallet caps, and updating liquidity swap parameters. If the owner private key is compromised or held by an untrusted malicious party, taxes can be set excessively high (creating a honeypot mechanism) or trading parameters can be restricted.

Fixing Remarks:

Remediation: Renounce ownership or transfer ownership privileges to a Multi-Sig Wallet (e.g., Safe with 3-of-5 threshold) with an enforced TimeLock contract (minimum 24–48 hour delay).

Code Patch:

```
    Enforce strict upper boundaries on administrative fee changes
function setFees uint256  buyFee  uint256  sellFee  external onlyOwner
    require  buyFee    1000, "Buy fee exceeds max 10%");
    require  sellFee <= 1000  "Sell fee exceeds max 10%");
    buyFee =  buyFee
    sellFee =  sellFee
}
```

CN-02 Key Findings

Category	Severity	Target	Status
Business Logic	Medium	TokenDistributor	Acknowledged

Potential Unlimited Allowance In **TokenDistributor** Interactions

Vulnerability Analysis:

The **CRYSTAL STONES** logic authorizes maximum allowances (type(uint256).max) to the TokenDistributor contract or Uniswap/PancakeSwap routers to facilitate automated Liquidity Injection and BNB conversions. Infinite approvals can be dangerous if a DEX router endpoint is misconfigured or hijacked.

Fixing Remarks:

Remediation: Approve exact transactional allowances as needed during liquidity additions or execute programmatic reset calls.

Code Patch:

```
// Replace unlimited allowance with safe exact approval
function _approveIfNeeded(IERC20 token, address spender  uint256 amount) internal {
    uint256 currentAllowance = token allowance(address(this), spender);
    if (currentAllowance < amount) {
        token approve(spender  type(uint256).max);
    }
}
```

CN-03 Key Findings

Category	Severity	Target	Status
Business Logic	Medium	swapAndLiquify	Acknowledged

In Unchecked DEX Router Slippage & Reentrancy during **swapAndLiquify**

Vulnerability Analysis:

When auto-liquidity/tax swapping triggers inside _transfer(), calls to the DEX router (PancakeSwap) execute swapExactTokensForETHSupportingFeeOnTransferTokens. The amount out min parameters are typically set to 0 or calculated on-chain, exposing the automated sell sequence to sandwich attacks and MEV arbitrage front-running

Fixing Remarks:

Remediation: Utilize a reentrancy guard lock (inSwap) to ensure reentrancy cannot occur during external call transfers to router paths, and implement reasonable minimum output checks.

Code Patch:

```
// Ensure state lock protection during auto-swap
modifier lockTheSwap {
    inSwap = true;
    _;
    inSwap = false;
}

function swapTokensForEth uint256 tokenAmount  private lockTheSwap {
    // Router execution logic...
}
```

Vulnerability Scan

REENTRANCY

No reentrancy risk found

Programmatic Verification Script (Ethers.js / Web3.js)

Automated script snippet to audit holder balances directly from a BEP-20 node

Vulnerability Description

Additional Observations: More amount of the CRYSTAL STONES can **NOT** be minted by a private wallet or contract. **(This is Essentially normal for most contracts)**

Scanning Line:

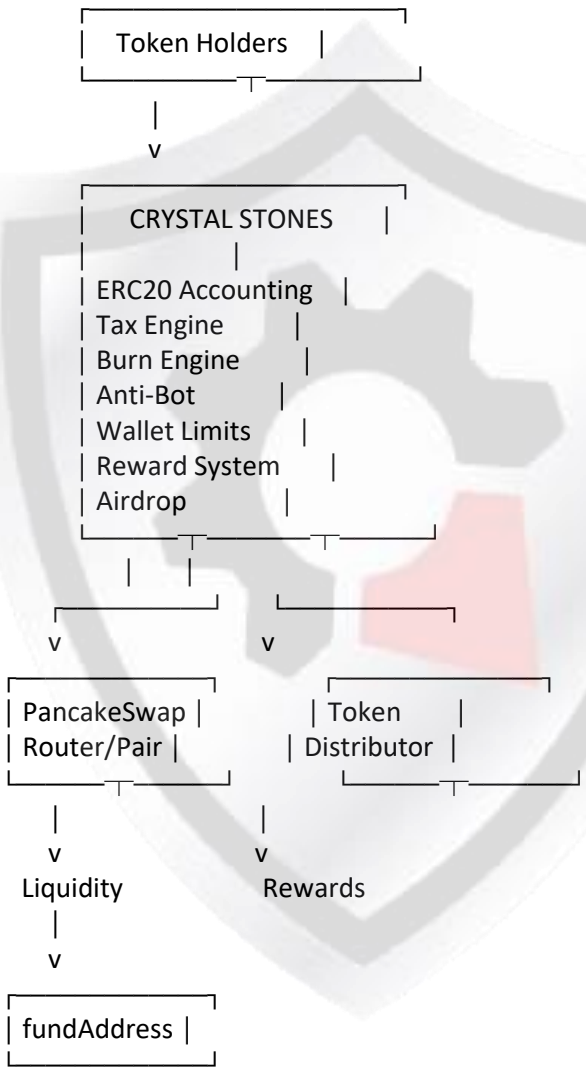
```
const { ethers } = require("ethers");
const TOKEN_ADDRESS = "0xe252FCb1Aa2E0876E9B5f3eD1e15B9b4d11A0b00";
const BSC_RPC = "https://bsc-dataseed.binance.org/";
const abi = [
  "function totalSupply() view returns (uint256)",
  "function balanceOf(address account) view returns (uint256)",
  "function decimals() view returns (uint8)"

async function runHolderAudit(topHoldersArra  ) {
  const provider = new ethers.JsonRpcProvider(BSC
  const contract = new ethers.Contract(TOKEN_ADDRESS, abi, provider
 const totalSupply = await contract.totalSupply();
  const decimals = await contract.decimals();
  console.log(`Total Supply: ${ethers.formatUnits(totalSupply, decimals)}`);
  for (let holder of topHoldersArray) {
    const balance = await contract.balanceOf(holder
    const percentage = (BigInt(balance) * 10000n / BigInt(totalSupply
    console.    (`Address: ${holder} | Balance: ${ethers.formatUnits(balance, decimals)} | Share:
${Number(percentage) / 100}%`);
  }
}
```

Vulnerability Description

Architecture Review

No reentrancy risk found

High-Level Architecture



The central architectural weakness is that **many economically significant components converge on privileged owner-controlled parameters**.

MANUAL REVIEW

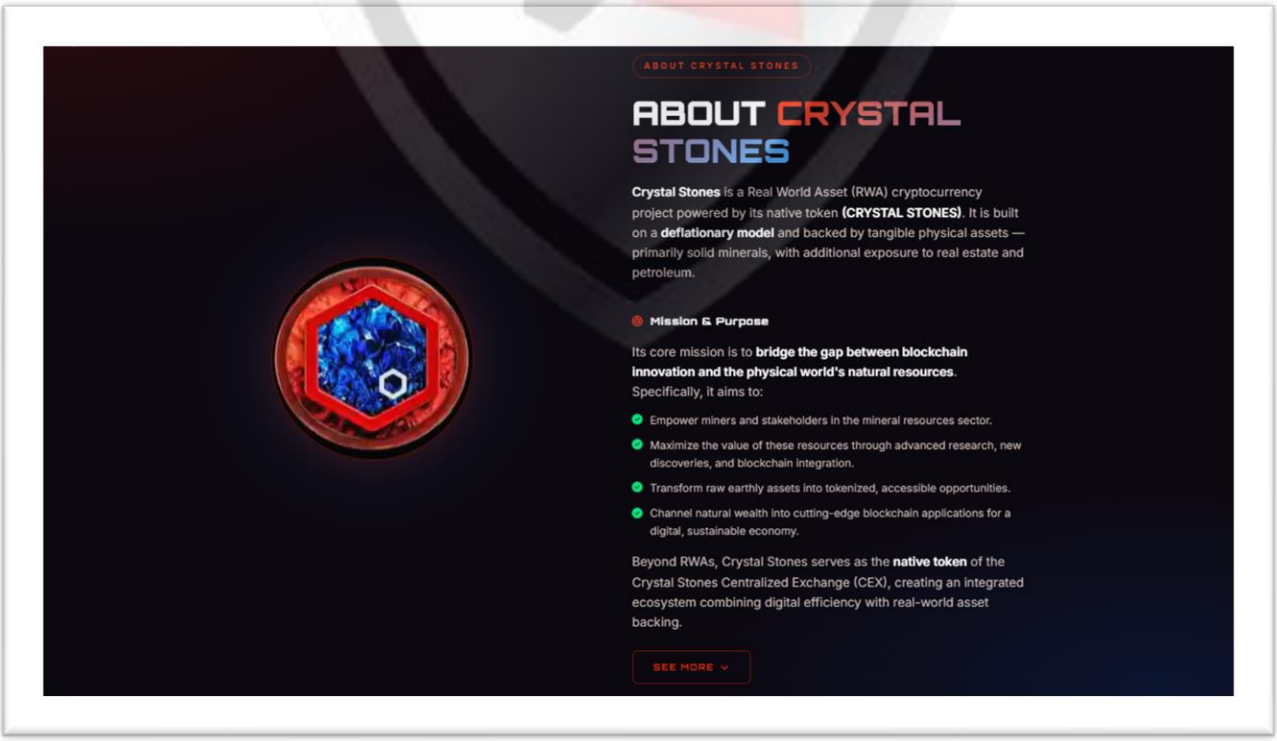
Crystal Stones is a Real World Asset (RWA) cryptocurrency project powered by its native token **(CRYSTAL STONES)**. It is built on a **deflationary model** and backed by tangible physical assets — primarily solid minerals, with additional exposure to real estate and petroleum.

TOKEN NAME: CRYSTAL STONE
Ticker: $CRYSTAL STONE

Chain/Standard: BINANCE NETWORK

LAUNGUGE: SOLIDITY

The CRYSTAL STONE Platform Is Launched On the Binance Network



Crystal Stones: Deployer

0xB8C30DC6414378c60E19fc2DC8D578488B071dD3

Audited Files

O.CRYSTALSTONES.sol

Contract Creator Address

0xB8C30DC6414378c60E19fc2DC8D578488B071dD3

Deployed Contracts:

0xe252FCb1Aa2E0876E9B5f3eD1e15B9b4d11A0b00

Creator TXH Contracts:

0x9c398f9c80ec18f03a01bee79beb6da962a282dbcb774807bd0c276391351f8e

Auto Contract Scan

Basic Info

Token Contract Address	0xe252...0b00
Owner (Renounced)	0x0000...0000
Total Supply	135,000,000
Issue Platform	fatsale

Risk Check

- Renounced, Slippage cannot be modified
- Doesr Anti whale is No More modifiable
- Contract Ownership Renounced
- Owner can not tamper with balance
- Doesn't look like a proxy contract
- No whitelist
- No blacklist
- Admin privileges abandoned
- Can not Mint
- Can not take back ownership
- No trading-cool-down mechanism

Mechanism Introduction

Buy Tax	3%
Sell Tax	6%
Additional	加池分红

Sell detection

Wallets	Success	Failed	Siphoned
5	5	0	0
Tax	Ave Tax 6.212% Tax 6%	Count 5	

Token Holders Info

Token Holders: 4299

Top10 ratio(exclude blackhole)	16.38%
1. Blackhole/黑洞地址1	30.98M (22.95%)
2. Cakev2: CRYSTAL STONES/WBNB	6.9M (5.11%)
3. 0x...36a7	3.28M (2.43%)
4. 0x...5dab	2.83M (2.09%)
5. 0x...bb67	2.69M (1.99%)
6. 0x...2776	2.2M (1.63%)
7. 0x...bf50	1.7M (1.26%)
8. 0x...2316	1.52M (1.13%)
9. 0x...6d18	1.47M (1.09%)
10. 0x...80a2	1.35M (1%)

More Details

LP

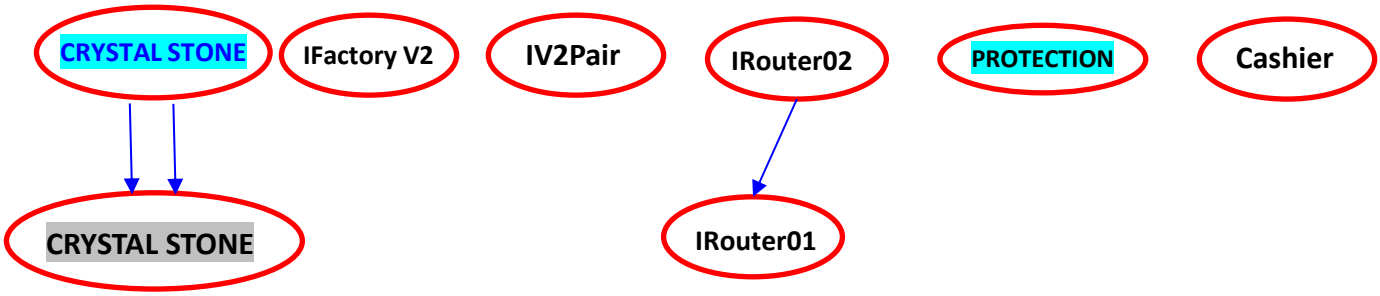
LP Holders: 7 — Total Supply: 16,160.7861

Percentage of LP locked: 97.35%

	Amount	Lock Date	Unlock Start	Unlock End	
1. Pinksale: PinkLock V2	11.76K	2026-06-18	2030-08-29	2030-08-29	11.76K (72.79%)
2. Fatsale	3,366.24	2023-09-07	2030-05-10	2030-05-10	3,366.24 (20.83%)
3. Blackhole/黑洞地址					426.82 (2.64%)
4. 0x...9706					231.25 (1.43%)
5. 0x...37bf					196.08 (1.21%)
6. Blackhole/黑洞地址					175.79 (1.09%)
7. 0x...5ea4					0.8 (0%)

More Details

INHERITANCE GRAPH



Identifier	Definition	Severity
CEN-12	Centralization privileges of CRYSTAL STONE	Medium

Vulnerability 0 : No important security issue detected.
Threat level: Low

```
477
478         uint bal = IERC20(tokenOther).balanceOf(address(mainPair));
479         isAdd = bal > r;
480     }
481
482     function _isRemoveLiquidity() internal view returns (bool isRemove) {
483         ISwapPair mainPair = ISwapPair(_mainPair);
484         (uint r0, uint256 r1, ) = mainPair.getReserves();
485
486         address tokenOther = currency;
487         uint256 r;
488         if (tokenOther < address(this)) {
489             r = r0;
490         } else {
491             r = r1;
492         }
493
494         uint bal = IERC20(tokenOther).balanceOf(address(mainPair));
495         isRemove = r >= bal;
496     }
```

ISSUES CHECKING STATUS

	Issue Description	Checking Status
1.	Compiler errors.	PASSED
2.	Race Conditions and reentrancy. Cross-Function Race Conditions.	PASSED
3.	Possible Delay In Data Delivery.	PASSED
4.	Oracle calls.	PASSED
5.	Front Running.	PASSED
6.	Sol Dependency.	PASSED
7.	Integer Overflow And Underflow.	PASSED
8.	DoS with Revert.	PASSED
9.	Dos With Block Gas Limit.	PASSED
10.	Methods execution permissions.	PASSED
11.	Economy Model of the contract.	PASSED
12.	The Impact Of Exchange Rate On the solidity Logic.	PASSED
13.	Private use data leaks.	PASSED
14.	Malicious Event log.	PASSED
15.	Scoping and Declarations.	PASSED
16.	Uninitialized storage pointers.	PASSED
17.	Arithmetic accuracy.	PASSED
18.	Design Logic.	PASSED
19.	Cross-Function race Conditions	PASSED
20.	Save Upon solidity contract Implementation and Usage.	PASSED
21.	Fallback Function Security	PASSED

AUDIT RESULT

PASSED

Identifier	Definition	Severity
CEN-02	Initial asset distribution	Minor

All of the initially minted assets are sent to the contract deployer when deploying the contract. This is Normal for most

deployer and/or contract owner .

Additional Observations

Liquidity Lock Status Audit

To ensure the DEX Liquidity Pair (PancakeSwap Pair 0x52bF227C8D43026697FA5Fa6Cd4a81C446977130) cannot be suddenly pulled by the deployer:

Identify the Liquidity Pair (LP) Token:

Locate the PancakeSwap V2 Pair address linked to the token (e.g., CRYSTAL STONES / WBNB pair).

Track LP Token Ownership:

Open the LP Token contract on BscScan and examine the **Holders** tab.

Burn Address Check: Verify if LP tokens were sent to 0x000000000000000000000000000000000000dEaD or 0x0000000000000000000000000000000000000000 (100% permanently destroyed liquidity).

Locker Contract Check: If LP tokens sit in a third-party locker (e.g., PinkLock, Unicrypt, Team Finance, StreamFlow):

Query the locker's smart contract state (getLockDetails or locks) via BscScan.

Verify the **Unlock Timestamp** (Unix time). Ensure it is set sufficiently far into the future (e.g., > 365 days).

Verify **Owner/Beneficiary:** Confirm the lock cannot be revoked early or withdrawn via an emergency bypass function in the locker contract.

RECOMMENDATION

Project stakeholders should be consulted during the initial asset distribution process.

RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-7 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The CRYSTAL STONES project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project

References

1 MITRE. CWE-1041: Use of Redundant Code. https://cwe.mitre.org/data/definitions/1041.html.

2 MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. https://cwe.mitre.org/data/definitions/1099.html.

3 MITRE. CWE-561: Dead Code. https://cwe.mitre.org/data/definitions/561.html.

4 MITRE. CWE-563: Assignment to Variable without Use. https://cwe.mitre.org/data/definitions/563.html.

5 MITRE. CWE-663: Use of a Non-reentrant Function in a Concurrent Context. https://cwe.mitre.org/data/definitions/663.html.

6 MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. https://cwe.mitre.org/data/definitions/837.html.

7 MITRE. CWE-841: Improper Enforcement of Behavioral Workflow. https://cwe.mitre.org/data/definitions/841.html.

8 MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

9 MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

10 MITRE. CWE CATEGORY: Concurrency. https://cwe.mitre.org/data/definitions/557.html.

11 MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

12 OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor

Smart contract is interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.

DISCLAIMERS

Vital Block Security provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way

to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT'S OR ANY OTHER INDIVIDUAL'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.

CERTIFICATE BY VITAL BLOCK SECURITY

VITALBlock SKYNET

ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 8+ casual contributors.

1000+
Audits Completed

$12B+
Secured

1M+
Lines of Code Audited

@vb_audit

@Vitalblock

@vitalblock

github.com/VBS-LABS

Info@vitalblock.org

www.vitalblock.org

Blockchain Security | Smart Contract Audit | KYC Certification | SAFU .

MADE IN CANADA